

// Jivops

NGINX

Guía completa — 20 lecciones

Fundamentos · Proxy inverso y balanceo · Seguridad y TLS · Rendimiento y operación

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es NGINX y su arquitectura
- Lección 02 — Estructura de la configuración y contextos
- Lección 03 — Server blocks y server_name
- Lección 04 — Location y prioridad de coincidencia
- Lección 05 — Contenido estático: root, alias y try_files

Módulo 2: Proxy inverso y balanceo

- Lección 06 — Proxy inverso y cabeceras
- Lección 07 — Upstream y balanceo de carga
- Lección 08 — Fallos de backend y comprobaciones pasivas
- Lección 09 — Caché de proxy
- Lección 10 — WebSockets y timeouts del proxy

Módulo 3: Seguridad y TLS

- Lección 11 — HTTPS y TLS
- Lección 12 — Redirección a HTTPS y HSTS
- Lección 13 — Limitación de peticiones y conexiones
- Lección 14 — Control de acceso y autenticación
- Lección 15 — Cabeceras de seguridad y ocultar la versión

Módulo 4: Rendimiento y operación

- Lección 16 — Ajustes de rendimiento
- Lección 17 — Compresión y caché del navegador
- Lección 18 — Logs y formatos
- Lección 19 — Recarga sin cortes y troubleshooting
- Lección 20 — Proyecto final: NGINX listo para producción

Módulo 1: Fundamentos

Lección 01 — Qué es NGINX y su arquitectura

Un servidor web, proxy inverso y balanceador de alto rendimiento con arquitectura orientada a eventos: un proceso maestro lee la configuración y gestiona varios procesos worker, cada uno de los cuales atiende muchas conexiones a la vez sin crear un hilo por conexión.

Porque es orientado a eventos: pocos workers atienden muchas conexiones a la vez sin crear un proceso o hilo por conexión.

Lección 02 — Estructura de la configuración y contextos

En directivas agrupadas en contextos anidados: el principal (main), events, http y dentro de http los bloques server y location; una directiva se hereda de un contexto superior salvo que se redefina.

Para dividir la configuración en ficheros, por ejemplo uno por sitio, en lugar de un único fichero enorme.

Lección 03 — Server blocks y server_name

Primero por la dirección y puerto de listen, y después compara la cabecera Host con server_name; si ninguno coincide usa el server por defecto de ese listen (default_server o el primero definido).

Para no servir un sitio por error cuando llega una petición con un nombre de host no esperado.

Lección 04 — Location y prioridad de coincidencia

Primero las coincidencias exactas (=), después los prefijos con ^~, luego las expresiones regulares (~ y ~*) por orden de aparición y, si ninguna aplica, el prefijo más largo coincidente.

Hace que, si ese prefijo coincide, NGINX no evalúe las expresiones regulares posteriores.

Lección 05 — Contenido estático: root, alias y try_files

Con root la URI completa se añade a la ruta (root /var/www; location /img/ sirve /var/www/img/...), mientras que con alias la parte del location se sustituye por la ruta indicada; try_files prueba varias rutas y usa una alternativa si no existen.

Para servir el index.html cuando la ruta no corresponde a un fichero, de modo que el enrutado lo resuelva el frontend.

Módulo 2: Proxy inverso y balanceo

Lección 06 — Proxy inverso y cabeceras

Además de `proxy_pass`, suelen pasarse cabeceras como `Host`, `X-Real-IP` y `X-Forwarded-For` (`proxy_set_header`) para que la aplicación conozca el host original y la IP real del cliente; por defecto NGINX cambia `Host` por el del upstream.

Porque la conexión llega desde NGINX; la IP real solo se conoce si se pasa en `X-Forwarded-For` o `X-Real-IP`.

Lección 07 — Upstream y balanceo de carga

Definiendo un bloque upstream con varios servidores y usando `proxy_pass` hacia él; por defecto reparte por turnos (round robin), y se puede elegir `least_conn`, `ip_hash` o dar pesos con `weight`.

Cuando interesa que un mismo cliente vaya al mismo servidor, por ejemplo con sesiones locales.

Lección 08 — Fallos de backend y comprobaciones pasivas

De forma pasiva: marca un servidor como no disponible tras `max_fails` fallos durante `fail_timeout` y deja de enviarle tráfico durante ese tiempo; `proxy_next_upstream` define cuándo reintentar con otro servidor; los health checks activos son una función de NGINX Plus.

Que se dupliquen operaciones no idempotentes (como un pago) si el primer intento sí llegó a procesarse.

Lección 09 — Caché de proxy

Definiendo la zona con `proxy_cache_path` en el contexto http y activándola con `proxy_cache` en el location, con `proxy_cache_valid` para el tiempo por código de respuesta; así NGINX sirve respuestas repetidas sin consultar al backend.

Las personalizadas por usuario o con sesión, para no servir a un usuario datos de otro.

Lección 10 — WebSockets y timeouts del proxy

Usar HTTP/1.1 hacia el backend y pasar las cabeceras `Upgrade` y `Connection` (`proxy_http_version 1.1` y `proxy_set_header`), además de ajustar `proxy_read_timeout` para que las conexiones largas no se cierren por inactividad.

Porque `proxy_read_timeout` vale 60 s por defecto y se cierra si no hay tráfico; hay que aumentarlo o enviar latidos.

Módulo 3: Seguridad y TLS

Lección 11 — HTTPS y TLS

Con `listen 443 ssl`, indicando `ssl_certificate` y `ssl_certificate_key`, limitando los protocolos a TLS 1.2 y 1.3 (`ssl_protocols`) y, si se desea, activando HTTP/2; el certificado puede obtenerse y renovarse con herramientas como Let's Encrypt.

Porque son protocolos obsoletos con vulnerabilidades conocidas; se usan TLS 1.2 y 1.3.

Lección 12 — Redirección a HTTPS y HSTS

Con un server en el puerto 80 que devuelve una redirección 301 a `https://$host$request_uri` y, una vez comprobado que todo funciona, la cabecera Strict-Transport-Security (HSTS) para que los navegadores usen HTTPS directamente.

Que si el certificado falla o se quiere volver a HTTP, los navegadores que ya lo cachearon no podrán acceder durante mucho tiempo.

Lección 13 — Limitación de peticiones y conexiones

Con `limit_req_zone` (clave como `$binary_remote_addr`, tamaño de zona y tasa) definido en `http` y `limit_req` en el `server` o `location`, con `burst` para absorber ráfagas; `limit_conn` limita las conexiones simultáneas.

Para permitir ráfagas cortas por encima de la tasa, poniendo en cola las peticiones excedentes en lugar de rechazarlas de inmediato.

Lección 14 — Control de acceso y autenticación

Restricción por IP con `allow` y `deny`, autenticación básica con `auth_basic` y un fichero de usuarios, o delegar la decisión en un servicio externo con `auth_request`; siempre sobre HTTPS para no exponer credenciales.

Porque las credenciales van codificadas en base64, no cifradas, y en HTTP pueden interceptarse.

Lección 15 — Cabeceras de seguridad y ocultar la versión

Ocultar la versión con `server_tokens off` y añadir cabeceras de seguridad con `add_header`, como X-Content-Type-Options, X-Frame-Options o Content-Security-Policy, ajustadas a la aplicación.

Evita que el navegador adivine el tipo MIME y ejecute contenido como un tipo distinto del declarado.

Módulo 4: Rendimiento y operación

Lección 16 — Ajustes de rendimiento

`worker_processes` auto (un worker por núcleo), `worker_connections` para el máximo de conexiones por worker, `sendfile` para enviar ficheros sin copiarlos al espacio de usuario y `keepalive` hacia clientes y `upstream`, sin olvidar los límites de descriptores de fichero del sistema.

Porque reutiliza conexiones con el backend, evitando el coste de abrir una nueva por petición.

Lección 17 — Compresión y caché del navegador

Con `gzip on` y `gzip_types` para comprimir texto (HTML, CSS, JS, JSON), y con `Cache-Control` o `expires` para que el navegador reutilice los recursos estáticos; los ficheros con hash en el nombre pueden cachearse mucho tiempo.

Porque si el contenido cambia también cambia el nombre, así el navegador descarga la nueva versión sin problemas de caché antigua.

Lección 18 — Logs y formatos

Con `access_log` (peticiones, con el formato definido por `log_format`) y `error_log` (errores, con nivel), que conviene rotar y enviar a un sistema central; el formato puede incluir tiempos como `$request_time` y `$upstream_response_time`.

`$request_time` (tiempo total) y `$upstream_response_time` (tiempo del backend), para saber dónde se pierde el tiempo.

Lección 19 — Recarga sin cortes y troubleshooting

Validando con `nginx -t` y recargando con `nginx -s reload` (o `systemctl reload nginx`): el maestro lanza workers nuevos con la configuración nueva y los antiguos terminan sus conexiones; un 502 suele indicar backend caído y un 504, timeout del backend.

Que el backend esté en marcha y accesible, y el `error_log` de NGINX para ver el motivo de la conexión fallida.

Lección 20 — Proyecto final: NGINX listo para producción

HTTPS con TLS 1.2/1.3 y redirección desde HTTP, `server_tokens off` y cabeceras de seguridad, proxy con `upstream` y cabeceras correctas, `limit_req` en rutas sensibles, compresión y caché, logs rotados con tiempos, validación con `nginx -t` y monitorización.

HTTPS con redirección, límites de peticiones y monitorización con logs y alertas (más validación antes de recargar).